HirePulse — Interview Project Report

Project: HirePulse (AI Recruitment Management System)
Type: Full-stack web application
Live frontend: https://hirepulse-gamma.vercel.app
Live API: https://api-production-5f0fb.up.railway.app/api/v1

---

1. Elevator pitch

HirePulse is a full-stack AI recruitment platform where:

- **Candidates** upload resumes and apply to jobs
- **Recruiters / Admins** screen applications with AI, select or reject candidates, schedule interviews, and track status
- Status stays consistent across Candidate, Recruiter, and Admin dashboards

---

2. Problem it solves

Manual resume screening is slow and inconsistent. HirePulse centralizes:

1. Job and candidate management
2. AI-assisted resume-to-job matching (ATS / match scores)
3. Hiring decisions (Select / Reject)
4. Interview scheduling and status tracking
5. Role-based dashboards for Admin, Recruiter, and Candidate

---

3. End-to-end workflow

3.1 Candidate flow

1. Register / Login
2. Complete profile
3. Upload resume (PDF / DOCX / TXT)
4. Browse open jobs and apply
5. Track application status and interview status on the portal

3.2 Recruiter / Admin flow

1. Login
2. Manage companies, jobs, candidates, users (admin)
3. View applications
4. Run **AI Resume Screening** (compare resume vs job)
5. Review match score, ATS score, missing skills, strengths, suggestions
6. **Select / Reject** → confirmation modal → status saved in database
7. Schedule interview
8. Update interview status from dropdown
9. Application status syncs automatically (e.g. Interview Selected → Application Selected)

3.3 Pipeline diagram

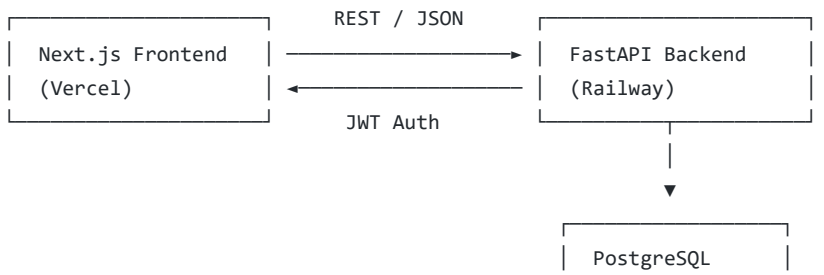
```
Candidate registers
        ↓
Uploads resume
        ↓
Applies to job  →  Application created
        ↓
Recruiter runs AI screening  →  ATS / match score + insights
        ↓
Select / Reject  →  Application status updated
        ↓
Interview scheduled
        ↓
Interview status changed (Completed / Selected / Rejected / …)
        ↓
Application status synced (single source of truth)
        ↓
Visible on Candidate + Recruiter + Admin dashboards
```

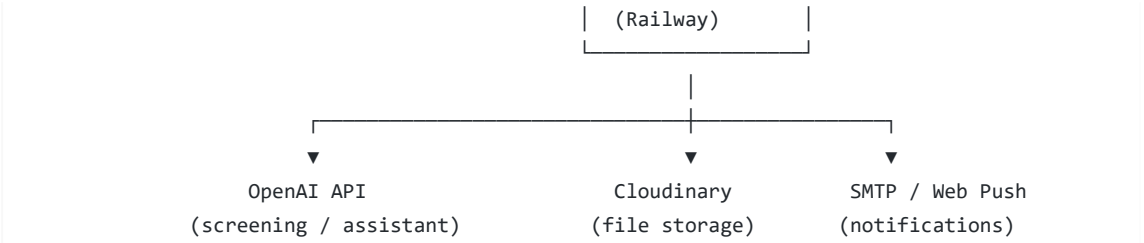
3.4 Application status labels (examples)

Status key	Display label
`applied`	Applied
`screening`	Under Review
`shortlisted`	Shortlisted
`interview`	Interview Scheduled
`interview_completed`	Interview Completed
`selected`	Selected
`rejected`	Rejected
`hired`	Hired
`offered`	Offered
`withdrawn`	Withdrawn

---

4. System architecture





Design notes:

- Frontend and backend are separated
- Backend is the source of truth for data and permissions
- JWT access + refresh tokens; role checks for `admin`, `recruiter`, `candidate`
- OpenAI is used when available; a **local screener fallback** keeps screening working if quota/API fails

---

5. Roles

Role	Capabilities
Candidate	Profile, resume upload, apply to jobs, view application & interview status
Recruiter	Jobs, screening, Select/Reject, interviews, applicants
Admin	Everything recruiters can do + users, recruiters, analytics, system-wide views

---

6. Technology stack

6.1 Frontend

Technology	Purpose
Next.js 15	React framework, App Router, Vercel deploy
React 19	UI library
TypeScript	Type safety
Tailwind CSS 4	Styling
Base UI / shadcn-style components	Dialogs, selects, buttons, forms
Axios	HTTP client + auth interceptors
React Hook Form + Zod	Forms and validation
Framer Motion	Animations
Recharts	Analytics charts
Lucide React	Icons
next-themes	Light / dark theme

6.2 Backend

Technology	Purpose
FastAPI	REST API framework
Uvicorn	ASGI server
SQLAlchemy 2	ORM
Alembic	Database migrations
PostgreSQL + psycopg2	Primary database
Pydantic / pydantic-settings	Validation + config
python-jose (JWT)	Access tokens
passlib + bcrypt	Password hashing
httpx	Outbound HTTP
pypdf / python-docx / striprtf	Resume text extraction
OpenAI Python SDK	AI parsing, matching, assistant
Cloudinary	Optional cloud file storage
pywebpush	Browser push notifications

6.3 Infrastructure

Technology	Purpose
GitHub	Source control
Vercel	Frontend hosting
Railway	API + PostgreSQL hosting
Docker Compose	Local Postgres for development

---

7. APIs and integrations

7.1 Internal REST API (`/api/v1`)

Area	Example endpoints
Auth	`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/me`
Resumes	`/resumes/me/upload`
Jobs	`/jobs`
Applications	`/applications/apply`, `/applications/compare`, `/applications/{id}/status`

Interviews	`/interviews`, `/interviews/{id}/status`
Candidates / Users	`/candidates`, `/users`
Analytics	`/analytics/overview`
Notifications	`/notifications`
Assistant	`/assistant/...`
Health	`/health`

7.2 External APIs / services

Service	Usage
OpenAI API (`gpt-4o-mini`)	Resume understanding, job–resume matching, AI assistant
Cloudinary	Resume / avatar cloud storage (fallback: local uploads)
SMTP	Email notifications (when configured)
Web Push (VAPID)	In-browser push notifications

7.3 AI fallback

If OpenAI is unavailable or over quota, screening continues via a **local multi-factor screener** (skills, experience, education, structure, etc.), so hiring is not blocked.

---

8. Key features to highlight

1. **Role-based access control** — Admin / Recruiter / Candidate
2. **AI resume screening** with ATS/match scores and explanations
3. **Select / Reject workflow** with confirmation modal + persistence
4. **Interview status management** with dropdown + timeline
5. **Single source of truth** for application status across all dashboards
6. **Notifications** (in-app; email/push when configured)
7. **Analytics dashboard** for hiring funnel insights
8. **Production deployment** on Vercel + Railway

---

9. Project layout (quick map)

```
backend/     FastAPI, Alembic, OpenAI, Cloudinary, business logic
frontend/    Next.js 15 admin + candidate portal UI
docs/        Deployment and project docs
docker/      Local Postgres setup
```

---

10. Local run (if asked)

```
# DB
docker compose up -d db

# API
cd backend && cp .env.example .env
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# Web
cd frontend && npm install && npm run dev
```

- Frontend: http://localhost:3000
- API health: http://localhost:8000/api/v1/health
- API docs (dev): http://localhost:8000/docs